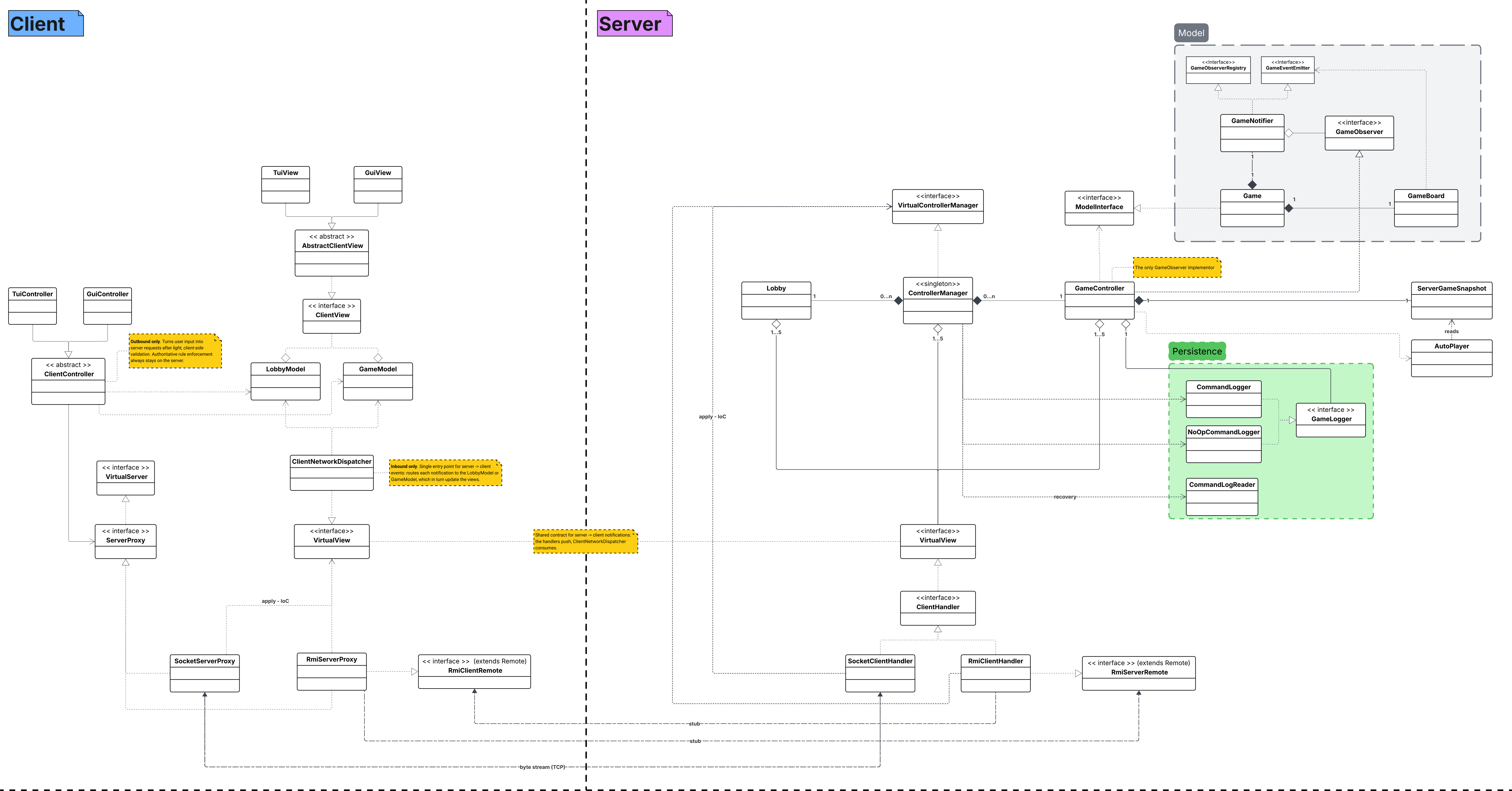




Architecture Overview — A simplified, high-level view of the client-server architecture, showing how the main components communicate. Attributes and methods are intentionally omitted and only the essential classes are shown; refer to the detailed Model, Server and Client class diagrams for the full design.

The architecture is deliberately **symmetric**, built around two granular contracts: VirtualServer / VirtualControllerManager for client -> server actions, and VirtualView for server -> client notifications. These interfaces are shaped directly on RMI's remote-method style, so the RMI transport realizes them natively as granular remote method calls (RmiServerRemote, RmiClientRemote). The Socket transport, which only exposes a raw byte stream, fills the gap: it encodes each call as a Command or Event record whose apply() re-dispatches it to the very same granular interface method (Inversion of Control).

Both transports therefore converge on the same high-level contracts — RMI directly, Socket through IoC records — and stay fully interchangeable.

The **Observer pattern** then decouples each end of the notification path from the network, so neither the domain model nor the UI depends on the wire. On the server the domain is observable: Game emits its changes through GameNotifier to its only observer, GameController (a GameObserver), which turns them into VirtualView notifications — the model never touches the network. On the client the local models are observable in turn: LobbyModel and GameModel notify their registered ClientView observers (TuiView / GuiView), which render. A single server-side change therefore travels Game -> GameNotifier -> GameController -> [transport] -> ClientNetworkDispatcher -> GameModel -> ClientView, with an Observer layer bracketing the symmetric transport at both ends.